

# ACTIVIDAD SEMANA 7

## Patrones de Diseño, Testing Unitario y Tipos de Datos Abstractos Lineales

|                                                                                                                                                                         |                                           |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------|
| <b>Estudiante:</b> Luis Alberto Villegas Merchan                                                                                                                        | <b>Carrera:</b> Programación Estructurada |
| <b>Materia:</b> Programación Orientada a Objetos                                                                                                                        | <b>Lenguaje:</b> Python 3.14              |
| <b>Repositorio GitHub:</b><br><a href="https://github.com/luisvillegas190/semana-7-entrega-final.git">https://github.com/luisvillegas190/semana-7-entrega-final.git</a> | <b>Fecha:</b> Septiembre 2026             |

### 1. Problema o Caso Práctico Seleccionado

Para esta actividad se seleccionó el problema de la **gestión de órdenes de despacho en una bodega**. Cuando los clientes o sucursales solicitan productos del inventario, los pedidos deben atenderse en el mismo orden en que fueron recibidos, sin adelantar a nadie de manera arbitraria. Este comportamiento responde de forma natural al principio **FIFO (First-In, First-Out)**, por lo que la estructura de datos ideal para resolver este caso es una **Cola (Queue)**.

### 2. Estructura de Datos Lineal Manual (Cola)

Siguiendo las instrucciones de la guía, la estructura fue implementada manualmente sin utilizar bibliotecas nativas como `collections.deque` o `queue.Queue`. En el archivo `estructuras_lineales.py` se creó la clase **ColaLineal** basada en **Nodos enlazados** (cada nodo almacena el dato y una referencia al siguiente nodo), manejando punteros al **frente** y al **final** de la estructura. Esto permite que las operaciones se ejecuten en tiempo constante **O(1)**:

| Operación             | Método                                       | Descripción                                                                |
|-----------------------|----------------------------------------------|----------------------------------------------------------------------------|
| Agregar elemento      | <code>encolar(elemento)</code>               | Inserta el elemento al final de la cola.                                   |
| Eliminar elemento     | <code>desencolar()</code>                    | Remueve y retorna el elemento al frente (FIFO). Lanza error si está vacía. |
| Consultar siguiente   | <code>ver_frente()</code>                    | Retorna el elemento del frente sin eliminarlo (Peek).                      |
| Verificar vacía       | <code>esta_vacia()</code>                    | Indica si la cola no contiene elementos.                                   |
| Cantidad de elementos | <code>tamano()</code> / <code>__len__</code> | Retorna la cantidad exacta de elementos almacenados.                       |

### 3. Aplicación del Patrón de Diseño Repository

El patrón **Repository** se utilizó para separar la forma en que se guardan y manipulan los datos de la lógica principal y de la interfaz gráfica. En el archivo `repositorio.py` se definieron dos interfaces con clases base abstractas (ABC):

- **IProductoRepository:** Se encarga de administrar los productos del catálogo (guardar, buscar, actualizar, eliminar y calcular métricas).

- **IDespachoRepository:** Administra los despachos de bodega. Su implementación **DespachoColaRepository** utiliza internamente la **ColaLineal manual**.

De esta forma, cuando se llama a `despachar_siguiente()`, el repositorio `desencola` la orden en orden de llegada y descuenta automáticamente las unidades físicas del inventario, asegurando que no se pueda despachar más stock del que realmente existe en bodega.

### 4. Evidencias de Pruebas Unitarias con Pytest

Se implementaron pruebas unitarias con **pytest** para comprobar el funcionamiento correcto de las operaciones principales (encolar, desencolar en orden FIFO, consultar el frente, guardar productos y despachar descontando stock). A continuación se muestra la evidencia de ejecución en consola de 5 pruebas clave:

```
PS C:\Users\ASUS\Desktop\Villegas_Luis_Semana7> python -m pytest -v tests/
===== test session starts =====
platform win32 -- Python 3.14.7, pytest-9.1.1
collected 5 items

tests/test_estructuras_lineales.py::TestColaLineal::test_encolar_un_elemento PASSED [ 20%]
tests/test_estructuras_lineales.py::TestColaLineal::test_encolar_multiples_y_orden_fifo PASSED [ 40%]
tests/test_estructuras_lineales.py::TestColaLineal::test_ver_frente_no_altera_la_estructura PASSED [ 60%]
tests/test_repositorio.py::TestProductoRepositoryMemoria::test_guardar_y_obtener_por_codigo PASSED [ 80%]
tests/test_repositorio.py::TestDespachoColaRepository::test_despachar_orden_fifo_y_descontar_stock PASSED [100%]

===== 5 passed in 0.05s =====
```

## 5. Enlace al Repositorio de GitHub e Instrucciones de Ejecución

### Enlace al repositorio público en GitHub:

<https://github.com/luisvillegas190/semana-7-entrega-final.git>

### Instrucciones para ejecutar el código:

- Instalar dependencias: `python -m pip install -r requirements.txt`
- Ejecutar pruebas unitarias: `python -m pytest -v`
- Ejecutar interfaz gráfica (Flet): `python main.py`
- Ejecutar demostración en consola: `python main.py --cli`